Ceres Django Application Structure

Overview

Ceres should be divided into Django applications based on clear areas of responsibility.

A Django app should own a coherent set of related concepts and behaviours. It should not necessarily correspond to a navigation item or an individual screen.

Recommended structure:

```
ceres/
├── config/
├── core/
├── accounts/
├── academics/
├── planning/
├── content/
├── collaboration/
├── files/
├── notifications/
└── search/
```

---

1. `config`

Purpose

Contains project-wide Django configuration.

This is not a feature app and should not contain product functionality.

Responsibilities

- Project settings
- Root URL configuration
- Application registration
- Environment configuration
- Project-level startup configuration

---

2. `core`

Purpose

Contains the overall Ceres experience and pages that combine information from several other apps.

Owns

Dashboard

- Daily schedule
- Deadlines
- Notes overview
- Quick actions
- Calendar overview
- Daily or weekly summary

Dashboard COULD features

- Assignment progress
- Study reminders
- Goals

Additional responsibilities

- Landing page
- Main application layout
- Shared navigation
- Shared page structure
- General error pages
- Product-wide contextual actions

Important rule

The dashboard should not own academic records, calendar events, notes or tasks.

It should retrieve and present information owned by other apps.

For example:

```
Dashboard
├── Schedule from Planning
├── Deadlines from Planning
├── Notes from Content
├── Assignments from Academics
```

```
├── Notifications from Notifications
└── Friends and projects from Collaboration
```

---

3. `accounts`

Purpose

Owns user identity, personal settings and user-to-user relationships.

Owns

Accounts

- Registration
- Authentication
- Account management
- User preferences
- Privacy preferences

User Profiles

- Name
- Profile image
- University
- Course
- Personal information
- Profile visibility

Friends

- Friend profiles
- Friend requests
- Accepted friendships
- Blocked or removed users
- Shared-content permissions

Potential future responsibilities

- Public profile customisation
- Academic identity
- User availability
- Presence information

Does not own

- Study groups
- Group projects
- Messaging
- Shared project content

Those belong to `collaboration`.

---

4. `academics`

Purpose

Owns the university-specific academic structure of Ceres.

Owns

Modules

- General module details
- Module membership
- Module resources
- Lists of associated lectures
- Lists of associated assignments

Lectures

- Lecture title
- Module relationship
- Date and time
- Basic room information
- Lecturer information
- Related assignments
- Links to associated notes
- Links to associated files

Lecture Hub

The Lecture Hub is a view of a lecture and its connected content.

It includes:

- Linked notes
- Linked attachments
- Related assignments

- Lecture details
- COULD: discussions

The Lecture Hub should not own separate note or attachment systems.

Timetable

- Weekly academic timetable
- Module entries
- Lecture entries
- Basic room information
- Links to Lecture Hub pages

Assignments

- Assignment details
- Module relationship
- Assignment deadline
- Assignment checklist or planner
- Group members
- Assignment views and filtering
- Upcoming assignments
- Active assignments
- Completed assignments

Assignment relationship with Tasks

Assignments should use the shared task-planning behaviour provided by `planning`.

Assignment priority, status and progress should not be recreated independently.

Revision — COULD

- Revision topics
- Confidence tracking
- Weak areas
- Flashcards
- Practice questions
- Mock exams
- Revision progress
- Links to study sessions

Recommended internal sections

```
academics/
├── modules
├── lectures
```

```
├── timetable
├── assignments
└── revision
```

These are conceptual areas inside one Django app, not separate Django apps.

Does not own

- General notes
- File storage
- Generic tasks
- Personal calendar events
- Group discussions

---

5. `planning`

Purpose

Owns time, scheduling, tasks, deadlines and study activity.

Owns

Calendar

- Day view
- Week view
- Month view
- Personal events
- Recurring events
- Shared calendars
- Deadline integration

Calendar COULD features

- Colour categories
- Academic event distinctions

Tasks

Tasks are a core shared planning engine.

They include:

- Quick task creation

- Priorities
- Categories
- Deadlines
- Recurring tasks
- Progress
- Assignees
- Task allocation

Tasks may be linked to:

- Assignments
- Modules
- Lectures
- Study sessions
- Group projects
- Notes

Study Sessions

- Create study sessions
- Invite participants
- Track time
- Set location
- Collaborate
- Link notes
- Link files and attachments
- Session history

Deadlines

Deadline behaviour should be centralised here.

Deadlines may come from:

- Assignments
- Tasks
- Calendar events
- Group projects
- Revision plans

Goals — potential module

- Academic goals
- Weekly goals
- Semester goals
- Goal progress

Habits — potential module

- Academic habits
- Recurring study habits
- Completion history
- Streaks

Important relationship

Assignments are academic objects, but their planning behaviour should use Tasks.

```
Assignment
└── Planning data
    ├── Tasks
    ├── Priority
    ├── Status
    ├── Progress
    └── Deadline
```

---

6. `content`

Purpose

Owns user-created academic content.

This prevents notes and whiteboards from becoming scattered across Academics, Collaboration and Planning.

Owns

Notes

- Rich text
- Markdown
- Images
- Tables
- Checklists
- Code blocks
- Search within notes
- Note organisation
- Folders or collections
- Tags
- COULD: mathematical notation

Note relationships

Notes may be linked to:

- Modules
- Lectures
- Assignments
- Study sessions
- Group projects
- Revision topics

Whiteboards

- Canvas
- Drawing
- Sticky notes
- Shapes
- Images
- Collaboration
- COULD: exporting

Meeting Notes

The reusable note system should support meeting-note templates.

Group Projects may create meeting notes, but the note content itself should remain owned by `content`.

Important rule

Do not create separate note systems for:

- Lectures
- Assignments
- Study sessions
- Group projects

Create one flexible Notes system that can be linked to each of them.

---

7. `collaboration`

Purpose

Owns shared spaces and communication between users.

Owns

Study Groups

- Group creation
- Membership
- Invitations
- Shared academic content
- Shared study activity

Group Projects

- Project workspace
- Members
- Project roles
- Shared files
- Shared notes
- Task boards
- Meeting notes
- Timeline
- Project progress
- Task allocation

Discussions — COULD

A single discussion system should support discussions attached to:

- Modules
- Lectures
- Assignments
- Group projects
- Study groups

Do not create separate discussion systems for each feature.

Messaging — COULD

- Direct messages
- Group conversations
- Replies
- Reactions
- File sharing
- Message search

Collaboration permissions

- Owners
- Members

- Editors
- Viewers
- Invite permissions
- Content sharing permissions

Group Project relationships

```
Group Project
├── Members from Accounts
├── Tasks from Planning
├── Notes from Content
├── Files from Files
├── Messages from Collaboration
└── Notifications from Notifications
```

Does not own

- User identity
- General files
- General notes
- Generic tasks

It coordinates those systems rather than recreating them.

---

8. `files`

Purpose

Owns uploaded files and reusable file-management behaviour.

Owns

File Management

- File storage
- File metadata
- File organisation
- Tags
- Search
- Preview
- Sharing
- Recent files

File relationships

Files may be linked to:

- Modules
- Lectures
- Assignments
- Notes
- Study sessions
- Group projects
- Messages

Attachments

Attachments should be relationships to Files rather than separate uploaded-file systems.

For example:

```
Lecture Hub attachment
→ File record

Assignment attachment
→ File record

Study session attachment
→ File record
```

Important rule

An uploaded file should exist once and be reusable in multiple places where appropriate.

---

9. `notifications`

Purpose

Owns alerts, reminders and notification preferences.

Owns

Notifications

- Assignment reminders
- Lecture reminders

- Calendar reminders
- Group updates
- Friend requests
- Message notifications
- Study-session invitations

Preferences

- Notification channels
- Notification categories
- Reminder timing
- Muted groups
- Read and unread state
- Configurable preferences

Notification sources

Other apps should generate notification events, but `notifications` should control how they are stored and presented.

```
Academics
→ Assignment reminder

Planning
→ Calendar reminder

Collaboration
→ Group update

Accounts
→ Friend request

Notifications
→ User notification feed
```

---

10. `search`

Purpose

Provides global search across the Ceres platform.

Owns

Generalised Search

Search across:

- Modules
- Lectures
- Notes
- Assignments
- Files
- Friends
- Messages
- Tasks
- Events

Search behaviour

- Search suggestions
- Grouped results
- Filtering
- Recent searches
- Search history
- Permission-aware results

Important rule

The Search app should not own the underlying content.

It should query content owned by other apps and combine the results.

Suggested result groups

```
Search Results
├── Academic
│   ├── Modules
│   ├── Lectures
│   └── Assignments
├── Content
│   ├── Notes
│   └── Files
├── Planning
│   ├── Tasks
│   └── Events
└── People and Collaboration
    ├── Friends
```

```
    ├── Groups
    └── Messages
```

Search may initially live in `core`, but a dedicated app is appropriate once global search becomes a major feature.

Complete Feature-to-App Mapping

Feature	Django app
Dashboard	`core`
Daily schedule display	`core`, sourced from `planning`
Deadline display	`core`, sourced from `planning`
Dashboard notes	`core`, sourced from `content`
Quick actions	`core`
Calendar	`planning`
Recurring events	`planning`
Personal events	`planning`
Shared calendars	`planning`
Timetable	`academics`
Modules	`academics`
Lectures	`academics`
Lecture Hub	`academics`
Assignments	`academics`
Assignment planning	`planning`
Assignment group members	`academics`, linked to `accounts`
Notes	`content`
Whiteboards	`content`
Revision	`academics`
Study sessions	`planning`
Friends	`accounts`

Feature	Django app
Profiles	`accounts`
Study groups	`collaboration`
Messaging	`collaboration`
Discussions	`collaboration`
Group projects	`collaboration`
Task boards	`collaboration`, using `planning` tasks
Meeting notes	`content`, linked to `collaboration`
Tasks	`planning`
Goals	`planning`
Habits	`planning`
Files	`files`
Attachments	`files`
Notifications	`notifications`
Global search	`search`
Placement Tracker	Future dedicated app or part of `academics`

Recommended Initial App Set

Create these at the beginning:

```
core
accounts
academics
planning
content
collaboration
files
notifications
```

Add `search` when global search development begins.

This avoids creating an empty Search app too early while keeping a clear destination for it later.

Potential Future App

`placements`

The Placement Tracker could eventually justify its own application.

It may include:

- Companies
- Applications
- Application stages
- Interviews
- Contacts
- Deadlines
- CV versions
- Offers

It should remain outside the initial app structure until the feature has a defined scope.

Recommended Dependency Direction

To prevent circular dependencies, use the following general direction:

```
accounts
   ↓
academics
planning
content
files
   ↓
collaboration
   ↓
notifications
search
core
```

Meaning:

- `accounts` is foundational.
- Academic, planning, content and file features may reference users.
- Collaboration combines those systems.
- Notifications respond to actions from all feature apps.
- Search reads from all feature apps.

- Core presents data from all feature apps.

Shared-System Rules

One Notes system

Use one Notes system across lectures, assignments, study sessions and group projects.

One Files system

Use one Files system across attachments, resources, messages and shared workspaces.

One Tasks system

Use one Tasks system across assignments, group projects and personal planning.

One Discussion system

Use one Discussion system that can attach to different academic or collaborative contexts.

One Notification system

All apps should generate notifications through the same notification system.

One Search experience

Global search should combine results rather than each feature creating an unrelated search page.

Final Proposed Structure

```
ceres/
├── config/
│
├── core/
│   ├── dashboard
│   ├── landing
│   └── shared_interface
│
├── accounts/
│   ├── authentication
```

```
│   ├── profiles
│   ├── preferences
│   └── friendships
│
├── academics/
│   ├── modules
│   ├── lectures
│   ├── timetable
│   ├── assignments
│   └── revision
│
├── planning/
│   ├── calendar
│   ├── tasks
│   ├── deadlines
│   ├── study_sessions
│   ├── goals
│   └── habits
│
├── content/
│   ├── notes
│   ├── whiteboards
│   └── content_organisation
│
├── collaboration/
│   ├── study_groups
│   ├── group_projects
│   ├── discussions
│   └── messaging
│
├── files/
│   ├── uploads
│   ├── organisation
│   ├── previews
│   └── sharing
│
├── notifications/
│   ├── notifications
│   ├── reminders
│   └── preferences
│
└── search/
    ├── global_search
    ├── filters
    └── search_history
```

This gives Ceres clear boundaries without turning every feature into a separate application.

The most important architectural decisions are:

1. Assignments belong to `academics`, but use task behaviour from `planning`.
2. Notes are owned by `content` and linked everywhere else.
3. Files are owned by `files` and linked everywhere else.
4. Group Projects belong to `collaboration`, but use tasks, notes and files from their owning apps.
5. The Dashboard belongs to `core`, but owns almost none of the data it displays.
6. Search and Notifications operate across all apps without owning the underlying feature data.